МЕТОДИЧЕСКИЕ УКАЗАНИЯ
к лабораторной работе № 1

Тема: «Настройка Visual Studio Code, виртуального окружения Python и работа с Git при отсутствии прав администратора»

Вариант А — базовая траектория · Вариант Б — расширенная траектория
Git используется в переносимой (portable) версии — установка не требуется

Дисциплина: «Программирование и прототипирование ИТ-решений»

Направление подготовки: 09.03.03 Прикладная информатика

Профиль: Бизнес и системная аналитика

Семестр / трудоёмкость: 1 семестр, 4 академических часа (2 пары)

Среда разработки: Visual Studio Code

Ограничение: права администратора отсутствуют

Форма контроля: отчёт о выполнении работы + устная защита

1. Цель работы

С нуля подготовить рабочее место разработчика в условиях, когда установка программ запрещена: развернуть переносимую (portable) версию Git, настроить Visual Studio Code, создать виртуальное окружение Python и заложить фундамент сквозного проекта «Мониторинг нефтяных скважин».

- развернуть и проверить Git;
- создавать и открывать проект в Visual Studio Code;
- создавать виртуальное окружение Python и выбирать его в VS Code;
- инициализировать Git-репозиторий и выполнять осмысленные коммиты;
- при необходимости связать локальный репозиторий с GitHub и корректно «отвязать» его после занятия.

2. Обязательное ядро (одинаково для обоих вариантов)

- Git установлен/развёрнут и проверен (git --version);
- проект открыт в VS Code и использует Python 3.11+ из виртуального окружения;
- исходный код размещён в каталоге src/ (структура src-layout);
- программа запускается и выводит результат по нефтегазовой тематике;
- файл .gitignore исключает служебные каталоги (.venv, __pycache__);
- в репозитории сделано не менее двух осмысленных коммитов;

- есть файлы README.md и pyproject.toml.

3. Глоссарий: все термины работы

Перед началом работы прочитайте термины. К ним можно возвращаться по ходу работы.

3.1. Python и командная строка

Термин	**Что это значит**
Интерпретатор	Программа, выполняющая код Python построчно. Команда python запускает именно её.
Терминал (командная строка)	Окно для ввода текстовых команд. В VS Code: Terminal → New Terminal (Ctrl + `).
Команда	Текст, вводимый в терминал и подтверждаемый клавишей Enter.
Переменная окружения	Параметр системы, влияющий на работу программ (PATH, PYTHONPATH).
PATH	Переменная, где система ищет исполняемые программы.
PYTHONPATH	Переменная, указывающая Python, где искать импортируемые модули.
Виртуальное окружение (venv)	Изолированная копия интерпретатора и пакетов для одного проекта.
Модуль	Отдельный файл .py с кодом.
Пакет	Каталог с модулями и файлом __init__.py.
Точка входа	Код, с которого начинается выполнение (if __name__ == "__main__").
Права администратора	Разрешение на установку программ. Portable-программам не требуется.

3.2. Git и GitHub

Термин	Что это значит
Система контроля версий	Запоминает историю изменений и позволяет вернуться к любой версии.
Portable-версия	Вариант программы без установки: просто папка, распакованная в любое место.
Репозиторий	Хранилище проекта, за которым следит Git (git init).
Рабочий каталог	Текущее состояние файлов на диске.
Индекс (staging area)	Промежуточная зона, куда попадают изменения после git add.
Коммит	Зафиксированный снимок изменений с сообщением.
Ветка	Независимая линия разработки.
Merge (слияние)	Объединение изменений одной ветки в другую.
Удалённый репозиторий (remote)	Копия репозитория на сервере (GitHub).
Push / Pull / Clone	push — отправить коммиты; pull — забрать изменения; clone — скачать репозиторий.
.gitignore	Файл со списком имён, которые Git не должен отслеживать.
GitHub	Веб-сервис для хранения Git-репозиториев.
Персональный токен доступа (PAT)	Пароль-ключ для входа из Git вместо обычного пароля.

3.3. Проект и VS Code

Термин	Что это значит
src-layout	Структура проекта, где весь код лежит в папке src/.

pyproject.toml	Файл с метаданными: имя, версия, описание, версия Python.
README.md	Текстовый файл с описанием проекта в формате Markdown.
Расширение VS Code	Дополнение, добавляющее функции в редактор.
settings.json	Настройки редактора для проекта (в т.ч. git.path).
launch.json	Конфигурация запуска и отладки через F5.
tasks.json	Описание задач, запускаемых Ctrl+Shift+B.
Точка останова	Отметка в коде, на которой программа останавливается.
Отладка	Пошаговое выполнение с контролем состояния.

4. Развёртывание Portable Git (без прав администратора)

Если прав администратора нет, обычный установщик не запустится. Используем переносимую (portable) версию: она не устанавливается в систему, а просто распаковывается в доступную папку.

> **Зачем это делаем:** *Portable-программа — это обычная папка с готовыми файлами. Ей не нужно писать в системные каталоги или реестр, поэтому прав администратора не требуется. Именно поэтому такой способ подходит для учебных аудиторий.*

Шаг 4.1. Скачать Portable Git

- откройте страницу релизов: https://github.com/git-for-windows/git/releases;
- найдите последний релиз (верхний) и разверните раздел Assets;
- скачайте файл вида PortableGit-x.y.z-64-bit.7z.exe (НЕ обычный установщик — он требует прав);
- сохраните файл в доступную папку, например «Загрузки».

> **Зачем это делаем:** *Файл с приставкой PortableGit — это самораспаковывающийся архив, а не установщик. Он просто разложит файлы в выбранную папку и не попросит прав администратора.*

Шаг 4.2. Распаковать

Запустите скачанный .exe и в качестве папки распаковки укажите:

```
C:\Users\ВашеИмя\PortableGit
```

Зачем это делаем: *Папка должна лежать там, куда у вас есть доступ на запись (обычно в вашем профиле C:\Users\ВашеИмя). Внутри появятся папки bin, cmd и другие — это и есть весь Git.*

Шаг 4.3. Сделать git доступным из терминала

Portable-версия не прописывается в системный PATH автоматически, поэтому путь указываем вручную. Достаточно одного из трёх способов.

Способ 1 (рекомендуется): PATH на текущую сессию

```
$env:PATH = "C:\Users\ВашеИмя\PortableGit\cmd;$env:PATH"
```

Зачем это делаем: *Эта команда добавляет папку с git в начало переменной PATH, но только в текущем окне терминала. Системные настройки не меняются, права не нужны. В новом терминале команду нужно повторить.*

Способ 2: указать git в настройках VS Code

Создайте файл .vscode/settings.json и добавьте строку с полным путём к git (обратные слэши в JSON удваиваются):

```
{
    "git.path": "C:\\Users\\ВашеИмя\\PortableGit\\bin\\git.exe"
}
```

Зачем это делаем: *VS Code умеет сам находить git по параметру git.path. Тогда PATH настраивать не нужно, и это работает во всех терминалах сразу.*

Способ 3 (по желанию): пользовательская переменная PATH

- нажмите Win + Pause → Дополнительные параметры системы → Переменные среды;
- в ВЕРХНЕМ списке (переменные пользователя) выберите Path → Изменить → Создать;
- добавьте C:\Users\ВашеИмя\PortableGit\cmd и нажмите ОК;
- перезапустите VS Code.

Зачем это делаем: *Права администратора нужны только для НИЖНЕГО списка (системные переменные). Пользовательские переменные меняются свободно — так git станет доступен постоянно, а не только в одном окне.*

Шаг 4.4. Проверить

```
git --version
```

Зачем это делаем: *Проверка показывает, что git действительно доступен. Если команда не найдена — перезапустите VS Code или проверьте путь из способа 1/2.*

5. Необходимое программное обеспечение

Инструмент	Назначение	Проверка установки
Python 3.11+	интерпретатор языка	python --version
Переносимая версия Git (раздел 4)	система контроля версий	git --version
Visual Studio Code	среда разработки	запуск редактора
Расширение Python	подсветка, анализ, выбор интерпретатора	установка из Marketplace
Расширение Python Debugger	отладка (для варианта Б)	установка из Marketplace

6. Общая подготовка (после настройки Git)

Шаг 6.1. Проверка инструментов

```
python --version
git --version
```

Зачем это делаем: *Записываем обе версии в отчёт. Это подтверждает, что окружение готово, а также пригодится, если что-то пойдёт не так — по версии проще искать причину.*

Шаг 6.2. Установка расширений VS Code

Панель расширений (Ctrl + Shift + X): установите Python и Python Debugger (оба — издатель Microsoft).

Зачем это делаем: *Расширение Python добавляет подсветку синтаксиса, подсказки и выбор интерпретатора. Python Debugger нужен для отладки — он обязателен в варианте Б.*

7. Вариант А (базовый): «Паспорт нефтяной скважины»

7.1. Что мы делаем

Создаём программу, которая выводит паспорт скважины: название, месторождение, глубину и дебит. Вариант для тех, кто раньше мало работал с Python и Git.

Шаг 1. Создать проект в VS Code

```
mkdir well-monitor-basic
```

Зачем это делаем: *Папка — это «дом» проекта: в ней будут лежать все его файлы. Команда mkdir (make directory) создаёт такую папку.*

Откройте папку в VS Code: File → Open Folder → well-monitor-basic, и создайте структуру файлов (кнопка New File на панели Explorer):

```
well-monitor-basic/
├── .gitignore
├── README.md
├── pyproject.toml
└── src/
    └── well_monitor/
        ├── __init__.py
        └── main.py
```

Зачем это делаем: *Структура src-layout отделяет код (папка src) от служебных файлов (README, настройки). Так проект остаётся аккуратным и его проще развивать на следующих работах.*

Шаг 2. Создать виртуальное окружение

```
python -m venv .venv
```

Зачем это делаем: *Виртуальное окружение изолирует библиотеки именно этого проекта. Если в разных проектах нужны разные версии одних и тех же библиотек, они не будут мешать друг другу. Папка .venv создаётся командой venv.*

```
.\.venv\Scripts\Activate.ps1
```

Зачем это делаем: *Активация «включает» окружение: все команды python в этом терминале начинают использовать интерпретатор из .venv. На это указывает надпись (.venv) в начале строки.*

Выберите интерпретатор: Ctrl + Shift + P → Python: Select Interpreter → .venv.

Зачем это делаем: *VS Code нужно явно указать, какой Python использовать для подсветки и запуска, иначе он может взять системный.*

Шаг 3. Написать код программы

Файл src/well_monitor/__init__.py:

```
"""Учебный проект мониторинга нефтяных скважин."""

__version__ = "0.1.0"
```

Зачем это делаем: *Наличие __init__.py превращает обычную папку well_monitor в пакет Python. Теперь её можно импортировать через from well_monitor import .... Строка документации (тройные кавычки) описывает назначение модуля.*

Файл src/well_monitor/main.py — точка входа:

```
"""Вывод паспорта условной нефтяной скважины."""

from well_monitor import __version__


WELL_NAME = "Скважина № 101"
FIELD_NAME = "Учебное месторождение"
DEPTH_M = 2850
DAILY_RATE_T = 12.5


def main() -> None:
    print("ПАСПОРТ НЕФТЯНОЙ СКВАЖИНЫ")
    print("-" * 32)
    print(f"Проект: Well Monitor v{__version__}")
    print(f"Название: {WELL_NAME}")
    print(f"Месторождение: {FIELD_NAME}")
    print(f"Глубина: {DEPTH_M} м")
    print(f"Дебит: {DAILY_RATE_T} т/сут")


if __name__ == "__main__":
    main()
```

Зачем это делаем: *Данные скважины записаны в константы (заглавными буквами) — их легко найти и поменять. Строка if __name__ == "__main__" означает: запускать main() только если файл выполняется напрямую, а не импортируется из другого места.*

Шаг 4. Создать служебные файлы

Файл .gitignore — что Git не должен отслеживать:

```
.venv/
__pycache__/
```

```
*.py[cod]
.vscode/
```

Зачем это делаем: *Папка .venv большая и восстанавливается одной командой, а __pycache__ создаётся автоматически. Хранить их в репозитории бессмысленно — .gitignore говорит Git их игнорировать.*

Файл pyproject.toml — метаданные проекта:

```
[project]
name = "well-monitor-basic"
version = "0.1.0"
description = "Учебный проект мониторинга нефтяной скважины"
requires-python = ">=3.11"
```

Зачем это делаем: *pyproject.toml — официальный способ описать проект: как он называется, какая у него версия и какая версия Python нужна. Это помогает другим людям (и будущему вам) понять проект.*

Файл README.md заполняем ниже (Шаг 4а).

Шаг 4а. Заполнить README.md

README — это «визитка» проекта, которую первым читает любой, кто его откроет. Скопируйте такой шаблон (кодировка UTF-8):

```
# Well Monitor (базовый трек)

## Назначение
Учебный проект: вывод паспорта нефтяной скважины.
Лабораторная работа № 1 курса «Программирование
и прототипирование ИТ-решений».

## Требования
- Python 3.11 или новее

## Как запустить
1. Создать виртуальное окружение:

       python -m venv .venv

2. Активировать его:

       .\.venv\Scripts\Activate.ps1

3. Запустить программу:

       $env:PYTHONPATH="src"
       python -m well_monitor.main
```

```
## Выводимые показатели
- Название скважины
- Месторождение
- Глубина, м
- Дебит, т/сут
```

Зачем это делаем: *README.md пишется в формате Markdown: знак # делает заголовок, дефис — пункт списка. Раздел «Как запустить» — самое ценное: по нему другой человек (или вы через месяц) запустит проект без устных пояснений.*

Шаг 5. Запустить программу

```
$env:PYTHONPATH="src"
python -m well_monitor.main
```

Зачем это делаем: *Команда python -m well_monitor.main запускает модуль как программу. Но пакет лежит в папке src, которую Python по умолчанию не ищет, поэтому сначала задаём PYTHONPATH — переменную, где Python ищет модули.*

Ожидаемый вывод:

```
ПАСПОРТ НЕФТЯНОЙ СКВАЖИНЫ
--------------------------------
Проект: Well Monitor v0.1.0
Название: Скважина № 101
Месторождение: Учебное месторождение
Глубина: 2850 м
Дебит: 12.5 т/сут
```

Зачем это делаем: *Сверяем фактический вывод с ожидаемым: если он совпал, программа работает корректно.*

Шаг 6. Выполнить работу с Git

```
git init
git config --local user.name "Фамилия Имя"
git config --local user.email "student@example.com"
git add .
git commit -m "Создать каркас проекта"
```

Зачем это делаем: *git init создаёт репозиторий — Git начинает следить за папкой. git config записывает, кто автор коммитов (--local значит «только для этого проекта»). git add помещает файлы в индекс, git commit сохраняет их в историю.*

Внесите изменение — добавьте константу и её вывод:

```
WELL_STATUS = "В работе"
print(f"Статус: {WELL_STATUS}")
```

Зачем это делаем: *Так мы проверяем на практике весь цикл «изменил → сохранил → закоммитил», а не просто делаем один-единственный коммит.*

Второй коммит и просмотр истории:

```
git add .
git commit -m "Добавить статус скважины"
git log --oneline
```

Зачем это делаем: *git log показывает историю: хэши коммитов и их сообщения. Два коммита с понятными сообщениями — признак аккуратной работы.*

8. Вариант Б (продвинутый): «Рабочее место аналитика»

8.1. Что мы делаем

Готовим проект, как в настоящей команде: логика и настройки разнесены по файлам, запуск и отладка настроены через VS Code, история Git ведётся в отдельной ветке.

Шаг 1. Создать проект

```
mkdir well-monitor-advanced
```

```
well-monitor-advanced/
├── .gitignore
├── README.md
├── pyproject.toml
├── .vscode/
│   ├── settings.json
│   ├── launch.json
│   └── tasks.json
└── src/
    └── well_monitor/
        ├── __init__.py
        ├── config.py
        └── main.py
```

Зачем это делаем: *Отличие от варианта А — папка .vscode с настройками и отдельный файл config.py. Это делает проект готовым к командной работе: настройки среды хранятся в репозитории вместе с кодом.*

Шаг 2. Настроить VS Code (settings.json)

```
{
    "python.analysis.extraPaths": ["./src"],
    "python.terminal.activateEnvironment": true,
    "git.path": "C:\\Users\\ВашеИмя\\PortableGit\\bin\\git.exe",
    "editor.rulers": [88],
```

```
    "files.encoding": "utf8"
}
```

Зачем это делаем: *extraPaths подсказывает VS Code, где лежит наш пакет (папка src), поэтому подсветка и автодополнение работают. Строка git.path нужна именно для portable-версии Git — она сообщает VS Code, где лежит git (способ 2 из раздела 4).*

Шаг 3. Написать код проекта

Файл src/well_monitor/config.py — настройки отдельно от логики:

```
"""Конфигурация учебного проекта."""

PROJECT_NAME = "Well Monitor"
WELL_ID = "W-101"
FIELD_NAME = "Учебное месторождение"
DEPTH_M = 2850
DAILY_RATE_T = 12.5
```

Зачем это делаем: *Данные вынесены в отдельный файл, чтобы их было удобно менять, не трогая логику программы. Это привычка, полезная для аналитика: настройки отдельно, код отдельно.*

Файл src/well_monitor/__init__.py:

```
"""Инструменты мониторинга нефтяных скважин."""

__version__ = "0.1.0"
```

Файл src/well_monitor/main.py:

```
"""Консольный прототип мониторинга скважины."""

from well_monitor import __version__
from well_monitor.config import (
    DAILY_RATE_T,
    DEPTH_M,
    FIELD_NAME,
    PROJECT_NAME,
    WELL_ID,
)


def print_well_card() -> None:
    print("=" * 42)
    print(f"{PROJECT_NAME} v{__version__}")
    print("КАРТОЧКА ТЕХНОЛОГИЧЕСКОГО ОБЪЕКТА")
    print("=" * 42)
    print(f"Идентификатор: {WELL_ID}")
```

```
    print(f"Месторождение: {FIELD_NAME}")
    print(f"Глубина: {DEPTH_M} м")
    print(f"Дебит: {DAILY_RATE_T} т/сут")
    print("=" * 42)


if __name__ == "__main__":
    print_well_card()
```

Зачем это делаем: *Импорт from well_monitor.config импортирует константы, поэтому main.py остаётся коротким и понятным.*

Шаг 4. Создать виртуальное окружение

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Выберите интерпретатор .venv (Ctrl + Shift + P → Python: Select Interpreter).

Зачем это делаем: *То же, что и в варианте A: окружение изолирует зависимости проекта, а выбор интерпретатора сообщает VS Code, какой Python использовать.*

Шаг 5. Запуск и отладка (launch.json)

Создайте .vscode/launch.json:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Запуск Well Monitor",
            "type": "debugpy",
            "request": "launch",
            "module": "well_monitor.main",
            "cwd": "${workspaceFolder}",
            "env": {
                "PYTHONPATH": "${workspaceFolder}/src"
            },
            "console": "integratedTerminal"
        }
    ]
}
```

Зачем это делаем: *launch.json сохраняет команду запуска, чтобы не вводить её вручную. Он же включает отладчик: теперь программу можно запускать клавишей F5 и ставить точки останова.*

Проверьте отладку: поставьте точку останова на строку в print_well_card (щёлчок слева от номера строки), нажмите F5, посмотрите значения переменных WELL_ID, DEPTH_M, DAILY_RATE_T, затем Continue (F5).

> **Зачем это делаем:** *Точка останова останавливает программу в заданном месте, чтобы увидеть, какие данные она обрабатывает. Это основной инструмент поиска ошибок.*

Шаг 6. Задача запуска (tasks.json)

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "Запустить Well Monitor",
            "type": "shell",
            "command": "python",
            "args": ["-m", "well_monitor.main"],
            "options": {
                "env": {
                    "PYTHONPATH": "${workspaceFolder}/src"
                }
            },
            "group": { "kind": "build", "isDefault": true },
            "problemMatcher": []
        }
    ]
}
```

Запуск задачи: Terminal → Run Build Task или Ctrl + Shift + B.

> **Зачем это делаем:** *tasks.json описывает повторяющиеся команды как задачи. Запуск по горячей клавише быстрее и не даёт забыть длинную команду.*

Шаг 7. Заполнить README как инструкцию

README должен быть настолько подробным, чтобы другой студент запустил проект без подсказок. Пример:

```
# Well Monitor (расширенный трек)

## Назначение
Учебный проект мониторинга нефтяной скважины с настроенным
рабочим местом разработчика.

## Требования
- Python 3.11+
- Visual Studio Code с расширениями Python и Python Debugger
- Git
```

```
## Создание окружения
    python -m venv .venv
    .\.venv\Scripts\Activate.ps1

## Запуск из терминала
    $env:PYTHONPATH="src"
    python -m well_monitor.main

## Запуск через VS Code
Нажмите F5 (конфигурация в .vscode/launch.json).

## Отладка
Поставьте точку останова в main.py и нажмите F5.

## Структура проекта
    src/well_monitor/  — исходный код
    .vscode/           — настройки VS Code
    pyproject.toml     — метаданные проекта
```

Шаг 8. Работа в ветке Git

```
git init
git switch -c feature/well-card
```

Зачем это делаем: *Ветка — это отдельная линия разработки. Работая в feature/well-card, мы не трогаем основную ветку, поэтому код в ней всегда остаётся рабочим.*

```
git add .
git commit -m "Создать структуру проекта"

git add .
git commit -m "Добавить конфигурацию скважины"

git add .
git commit -m "Настроить запуск и отладку в VS Code"
```

Зачем это делаем: *Несколько коммитов по чуть-чуть удобнее одного большого: по истории видно, что и когда менялось.*

```
git switch -c main
git merge feature/well-card
git log --oneline --graph
```

Зачем это делаем: *git merge объединяет готовую ветку с основной. Теперь вся работа есть в ветке main, а история аккуратная.*

9. Работа с GitHub в общей аудитории

За компьютером после вас будут работать другие студенты, поэтому в конце занятия полностью «отвяжите» свой аккаунт и обязательно выйдите из него в браузере.

9.1. Создание учётной записи GitHub

- откройте https://github.com и нажмите Sign up;
- введите почту, придумайте пароль и имя пользователя (username);
- подтвердите почту по ссылке из письма.

9.2. Локальная конфигурация вместо глобальной

```
git config --local user.name "Имя"    # только ТЕКУЩИЙ репозиторий
git config --global user.name "Имя"   # ВСЕ репозитории компьютера
```

Зачем это делаем: *--local действует только на текущий проект, --global — на весь компьютер. В общей аудитории используем --local, чтобы не оставлять свои данные следующим студентам.*

9.3. Создать удалённый репозиторий и связаться

На GitHub: New repository → имя well-monitor-basic → создать ПУСТОЙ репозиторий (без файлов).

```
git config --local user.name "Ваше Имя"
git config --local user.email "you@example.com"
git remote add origin https://github.com/ВАШ_ЛОГИН/well-monitor-basic.git
git push -u origin main
```

Зачем это делаем: *git remote add связывает локальный репозиторий с облачным (origin — имя этой ссылки). git push отправляет коммиты на GitHub.*

9.4. Вход с помощью токена (PAT)

- на GitHub: фото профиля → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token;
- задайте имя, срок действия и право repo;
- скопируйте токен (показывается один раз);
- при push введите логин как имя пользователя и токен как пароль.

Зачем это делаем: *GitHub не принимает обычный пароль при push — только токен. Токен сохраняется в хранилище Windows, поэтому в конце занятия его нужно удалить.*

9.5. «Отвязать» аккаунт и выйти из GitHub в конце

```
git remote remove origin
```

Зачем это делаем: *Удаляем связь локального репозитория с облаком, чтобы следующий студент не отправил свои коммиты в ваш аккаунт.*

Удалите сохранённый токен:

- Панель управления → Диспетчер учётных данных → Учётные данные Windows → удалите git:https://github.com;
- либо выполните git credential-manager erase.

```
git config --local --unset user.name
git config --local --unset user.email
```

Если пользовались SSH-ключом — удалите его на GitHub (Settings → SSH and GPG keys) и файлы из C:\Users\<пользователь>\.ssh.

Не забудьте выйти из учётной записи GitHub в браузере

- на сайте https://github.com нажмите на свою аватарку в правом верхнем углу;
- в открывшемся меню выберите пункт Sign out (выход из учётной записи).

Зачем это делаем: *Выход из учётной записи в браузере — обязательный шаг на общем компьютере. Даже если Git «отвязан», браузер всё ещё может помнить вашу сессию, и следующий студент попадёт в вашу учётную запись GitHub.*

Проверка: git remote -v должен выводить пустой результат, а страница github.com — показывать кнопку Sign in (значит, вы вышли).

10. Дополнительные упражнения (если осталось время)

Упражнение 1. Просмотр изменений

```
git status
git diff
```

Зачем это делаем: *git status показывает текущее состояние, git diff — что именно изменилось в коде. Это помогает понять, что будет в следующем коммите.*

Упражнение 2. Отмена ошибочной правки

```
git restore main.py
```

Убрать файл из индекса:

```
git restore --staged main.py
```

Зачем это делаем: *git restore возвращает файл к последнему сохранённому состоянию — полезно, если изменения получились неудачными.*

Упражнение 3. Ветки (для обоих вариантов)

```
git switch -c experiment
# внесите правку и закоммитьте её
git commit -am "Эксперимент"
git switch main
git log --oneline --graph
```

Упражнение 4. Проверка .gitignore

Добавьте строку *.tmp в .gitignore, создайте example.tmp и убедитесь, что Git его не видит:

```
git status
```

Упражнение 5. Просмотр конкретного коммита

```
git log -p
git show <хэш_коммита>
```

11. Пример оформления отчёта о работе

- Титульная часть: дисциплина, тема, ФИО, группа, дата, вариант;
- Цель работы;
- Оборудование и ПО с версиями (включая версию Git);
- Ход работы: шаги со скриншотами и пояснениями;
- Результаты: вывод программы;
- Вывод.

Пример заполненного отчёта (вариант А):

```
---------------------------------------
Отчёт по лабораторной работе № 1
Дисциплина: Программирование и прототипирование ИТ-решений
Вариант А (базовый)

1. Цель: настроить VS Code, создать venv, освоить Git
   на примере «Паспорт нефтяной скважины».

2. Оборудование и ПО: Python 3.12.2, Git 2.45.1,
   VS Code 1.98, расширения Python и Python Debugger.

3. Ход работы:
   - создал(а) структуру проекта;
   - настроил(а) venv и выбрал(а) интерпретатор;
```

```
   - написал(а) main.py и запустил(а) программу;
   - создал(а) .gitignore, pyproject.toml, README.md;
   - сделал(а) два коммита.

4. Результаты: программа выводит паспорт скважины;
   история Git содержит два коммита.

5. Вывод: освоил(а) настройку окружения и базовый Git.
----------------------------------------
```

12. Частые ошибки и их решение

Симптом	**Причина и решение**
git не является внутренней или внешней командой	Portable Git не распакован или путь не указан — см. раздел 4 (PATH или git.path).
PATH задан, но новый терминал git не видит	Команда $env:PATH действует только в текущем окне — повторите или настройте git.path.
Установщик просит права администратора	Скачан обычный установщик вместо portable — нужен PortableGit-x.y.z-64-bit.7z.exe.
python не является внутренней или внешней командой	Python не установлен или не в PATH. Переустановите, отметив Add Python to PATH.
Activate.ps1 не может быть загружен	Выполните Set-ExecutionPolicy -Scope CurrentUser RemoteSigned и повторите.
ModuleNotFoundError: No module named 'well_monitor'	Не задан PYTHONPATH или запуск не из корня. Выполните $env:PYTHONPATH="src".
git commit: Please tell me who you are	Не заданы user.name и user.email — выполните git config --local.
Точка останова не срабатывает	Не установлен Python Debugger или запуск не через F5.
При push просит пароль и отклоняет его	Нужен токен (PAT), а не обычный пароль — раздел 9.4.